

Лабораторна робота № 4

Вивчення системи переривань мікроконтролерів PSoC 6

Мета роботи: Ознайомитися з архітектурою переривань мікроконтролерів PSoC 6, джерелами і типами переривань, налаштуванням переривань з допомогою середовища розробки PSoC Creator 4.4, реалізувати проекти з використанням переривань.

Теоретичні відомості

Сімейство мікроконтролерів PSoC 6 підтримує переривання та системні винятки (виключення) на процесорних ядрах Cortex-M4 та Cortex-M0+. Будь-яка умова, яка зупиняє нормальне виконання команд, трактується процесором як виняток. Таким чином, *запит на переривання трактується як виняток*. Однак будемо вважати, що *переривання стосуються тих подій, які генеруються зовнішніми периферійними пристроями*, такими як таймери, блок послідовного зв'язку та сигнали виводів портів. *Винятки стосуються тих подій, які генеруються процесором*. До них відносяться помилки доступу до пам'яті та події внутрішнього системного таймера. Як переривання, так і винятки призводять до припинення виконання програми, а процесор виконує обробку винятків або процедуру обслуговування переривань (ISR – Interrupt Service Routine). Ядра Cortex-M4 і Cortex-M0+ забезпечують власну єдину таблицю векторів винятків як для обробників переривань (ISR), так і для обробників винятків.

Мікроконтролер PSoC 6 підтримує наступні функції переривання:

- підтримує 147 системних та периферійних переривань:
 - підтримує до 147 переривань Cortex-M4;
 - до 32 запитів переривань Cortex-M0+;
 - до 41 джерела переривань, здатних вивести пристрій з режиму глибокого сну (Deep Sleep);
- контролер вкладених векторних переривань (NVIC) інтегрований з кожним ядром ЦП, забезпечує незначну затримку переривань;
- контролер переривання пробудження (WIC - Wakeup interrupt controller) дозволяє виявити переривання (пробудження ЦП) в режимі глибокого сну;
- векторна таблиця може бути розміщена як у флеш пам'яті, так і в SRAM;
- рівні пріоритету, які налаштовуються (вісім рівнів для Cortex-M4 і чотири рівні для Cortex-M0+) для кожного переривання;
- сигнали переривань як по рівню, так і по фронту.

У ядрі Cortex-M4 джерело переривання системи 'n' безпосередньо підключено до IRQn. Для ядра Cortex-M0+, яке має лише 32 IRQ, джерело переривання, підключене до конкретного IRQn, може бути налаштоване, і будь-яке із 147 системних переривань може бути підключене до будь-якого IRQn.

NVIC забезпечує включення/ виключення окремих IRQ-кодів переривання, вирішення пріоритетів та зв'язок з ядром ЦП.

Кожне переривання може знаходитися в одному з наступних станів:

- Inactive (неактивний): не активний і не прийнятий.
- Pending (в очікуванні): чекає обслуговування процесором.
- Active (активний): обслуговується процесором, але обслуговування не завершено.
- Active and pending (активний і в очікуванні) обслуговується процесором, але в той же час очікується виключення від того ж джерела.

Блок Cortex-M4 WIC підтримує до 41- го джерела переривання, які можуть "розбудити" ЦП з режиму глибокого сну. Відповідно блок Cortex-M0+ WIC підтримує до 8- ми переривань. Пристрій виходить з режиму глибокого сну, як тільки один процесор прокидається. Блоки синхронізації синхронізують переривання з тактовою частотою процесора, оскільки периферійні переривання можуть бути асинхронними з тактовою частотою процесора.

Якщо NVIC отримує запит на переривання під час обслуговування іншого переривання або отримує декілька запитів на переривання одночасно, він оцінює пріоритет всіх цих переривань, пересилаючи номер запиту переривання з самим високим пріоритетом в ЦП. Отже, переривання з більш високим пріоритетом може блокувати виконання обробки переривання ISR з більш низьким перериванням.

Розглянемо послідовність налаштування переривань в мікроконтролері PSoC 6. Ці кроки є однаковими для ядер CM0+ і CM4, якщо не вказано інше, і повинні виконуватися для кожного ядра окремо.

1. Після скидання мікроконтролера всі переривання заборонені (відключені), і пріоритети переривань встановлені в нуль.

2. Налаштувати рівень пріоритету необхідного запиту IRQn в NVIC.

3. Налаштувати шлях переривання. Вибрати джерело переривання, яке підключено до потрібного запиту IRQn ЦП. Для CM0+ вибрати відповідне периферійне переривання для підключення до ЦП. Для CM4 це неможливо налаштувати. Джерело переривання n завжди підключено до IRQn.

4. Налаштувати джерело переривання (периферійне) та дозволити це переривання.

5. Налаштувати векторну таблицю з адресою запиту ISRn вектора. Векторна таблиця зберігає вхідні адреси кожного обробника винятків.

6. Необов'язково: очистити очікувані стани переривання в NVIC. Якщо дозволити попередньо заборонене переривання, то потрібно очистити стан очікування NVIC перед тим, як включити переривання. Це запобігає будь-якому помилковому запуску, викликаному попередніми перериваннями, які створили стан очікування.

7. Дозволити переривання в NVIC.

8. Дозволити глобальні переривання. Конфігурація переривання завершена.

Дозволене переривання спрацьовує, коли апаратний сигнал від джерела переривання активний і немає виконуваного переривання з більш високим пріоритетом. Коли це відбувається, ЦП переходить до місця в його таблиці векторів переривань, що відповідає спрацьованому перериванню. Це місце містить адресу ISRn, пов'язану з цим перериванням.

ISRn виконує завдання, необхідні для обробки переривання. Як правило, перше, що робить ISRn, - це очистка джерела переривання, щоб уникнути повторного входу в ISRn. Коли виконання ISRn припиняється, ЦП повертається до адреси команди в програмі, яку він виконував, до цього переривання.

Розглянемо програмні засоби, доступні для виконання описаних вище кроків.

Конфігурація переривань.

Використання PDL – це набір засобів розробки програмного забезпечення (SDK - software development kit), яке дозволяє розробляти програми (firmware) для пристроїв PSoC 6 MCU. Виклики функції PDL API використовуються для налаштування, ініціалізації, включення і використання периферійного драйвера. Одним з таких драйверів є системні переривання (SysInt). Системні переривання представляють структури та функції для налаштування і включення функції переривання. PDL також підтримують бібліотеки CMSIS-Core, які включають функції NVIC, використовувані для налаштування переривань.

В наступних кроках використовуються API- інтерфейси PDL та NVIC для налаштування переривання для запуску сигналу від периферійного пристрою.

1. Налаштувати периферію для генерування переривання. Наприклад, для GPIO налаштувати режим драйвера (з резистивним підтягуванням вгору або вниз), перервати генерацію сигналу по зростаючому або спадному фронтах та зняти маску переривання.

2. Налаштувати переривання за допомогою структури, наданої API SysInt. Структура визначена у файлі драйверів PDL SysInt cy_sysint.h.

Ця структура використовується для конфігурації наступного:

а) Джерела переривань (intrSrc):

- це виділені номери переривань, визначені в заголовочному файлі пристрою (наприклад: cy8c6347bzi_bld53.h).

- цей вибір залежить від того, якому ЦП хочемо назначити переривання.

- для ядра CM4 це число представляє як номер джерела переривання, так і номер IRQn ЦП. Виберіть номер периферійного переривання, яке потрібно направити до ЦП. Наприклад, для маршрутизації переривання 0-го порту GPIO потрібно призначити значення `ioss_interrupts_gpio_0_IRQn` (= 0).

- для ядра CM0+ це число представляє один з 32 мультиплексорів, доступних для маршрутизації переривання на CM0+. Оскільки кожен мультиплексор підключений до виділеної лінії CM0+IRQ, використовуйте це для вибору цільового номера CM0+IRQ. Наприклад, щоб використовувати мультиплексор №4 (CM0 + IRQ #4), потрібно вибрати `NvicMux4_IRQn` (= 4).

б) Номер переривання ядра CM0+ (cm0pSrc):

Методичні вказівки з курсу “Вбудовані системи опрацювання даних та управління на основі нейронних мереж”, 2025 р. Рабик В.

- цей параметр використовується тільки для CM0+.
- він представляє номер джерела переривання, який повинен бути перенаправлений до логіки переривань мультиплексора /генератора CM0+, вибрану за допомогою параметра `intrSrc`. Виберіть номер переривання периферійного переривання, яке потрібно направити до центрального процесора; наприклад, для маршрутизації переривання порту 0 GPIO, призначте значення "iOSS_interrupts_gpio_0_IRQn" (= 0).

в) Пріоритету переривання (`intrPriority`):

- встановити пріоритет переривання. Для ядра CM4 підтримуються пріоритети від 0 до 7. Для ядра CM0+ підтримувані пріоритети - від 0 до 3.

Примітки:

- В ядрі CM0+ деякі IRQ зарезервовані для використання програмним забезпеченням і не доступні для користувача.

- В ядрі CM0+ пріоритет переривання 0 зарезервований для системних викликів.

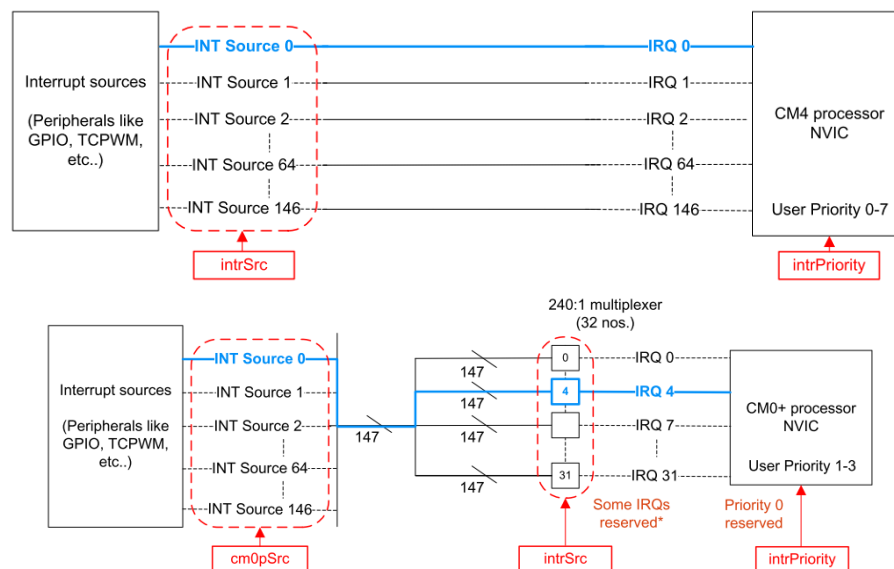


Рис. 4.1. Параметри структури PDL SysInt (виділені червоним кольором), які використовуються для конфігурації переривань.

Приклад шляху, який конфігурується, виділений на рис. 4.1 синім кольором.

3. Виклик `Cy_SysInt_Init(&SysInt_SW_cfg_1, ISR_1_handler)`.

Тут `SysInt_SW_cfg_1` – ім'я налаштованої структури. `ISR_1_handler` - ім'я обробника переривання, який виконується, при спрацюванні переривання. Ця функція виконує конфігурацію маршрутизації та пріоритету переривання, але не дозволяє її.

4. Виклик `NVIC_ClearPendingIRQ(SysInt_SW_cfg_1.intrSrc)`, щоб очистити будь-які очікувані переривання.

5. Виклик `NVIC_EnableIRQ(SysInt_SW_cfg_1.intrSrc)`, щоб дозволити переривання.

6. Виберіть функцію `__enable_irq()`, щоб увімкнути глобальні переривання. Це безпечно виконувати як перший крок, оскільки окремі переривання процесора ще не ввімкнено. Ви також можете виконати це пізніше, але переривання відключаються при запуску, якщо не викликано функцію глобального переривання.

Окрім драйвера PDL SysInt, API драйвера системних режимів живлення (SysPm) дозволяє функцію `Sleep-on-Exit`. Якщо в додатку поряд з перериваннями використовується режим "сну" або "глибокого сну", ця функція дозволяє внутрішньому програмному забезпеченню зберігати систему в режимі "сну" майже весь час, лише прокидається, щоб виконати переривання, а потім негайно повернутися до того ж режиму "сну". Програма не повертається до головної функції і залишається або в обробнику переривань, або в тому ж стані "сну", якщо функція `Sleep-on-Exit` знову не заборонена.

```
Cy_SysPm_SleepOnExit(true);
```

Конфігурування переривань з допомогою середовища розробки PSoC Creator 4.4.

Середовище PSoC Creator надає графічний інтерфейс для маршрутизації сигналів від периферійних пристроїв до лінії `IRQn` процесора. PSoC Creator надає компоненту переривання (SysInt). Ця компонента є елементом інтерфейсу користувача поверх драйвера SysInt PDL. Виходячи з конфігурації в компоненті, PSoC Creator генерує код для ініціалізації периферійних пристроїв, маршрутизації переривань та заповнення структури конфігурації переривання. Це зменшує об'єм коду, який ми повинні написати при налаштуванні переривань.

Використання графічного редактора (TopDesign).

Перетягніть компоненту із каталогу компонентів на TopDesign. Використовуйте TopDesign для розміщення та налаштування периферійних пристроїв, які забезпечують джерело переривання. Зверніться до опису (datasheet) компонента, щоб отримати інформацію про конфігурацію переривання певного периферійного пристрою. Деякі периферійні пристрої надають вивід переривання (наприклад, TCPWM). Помістіть екземпляр компонента SysInt і підключіть його до виводу переривання периферійного пристрою.

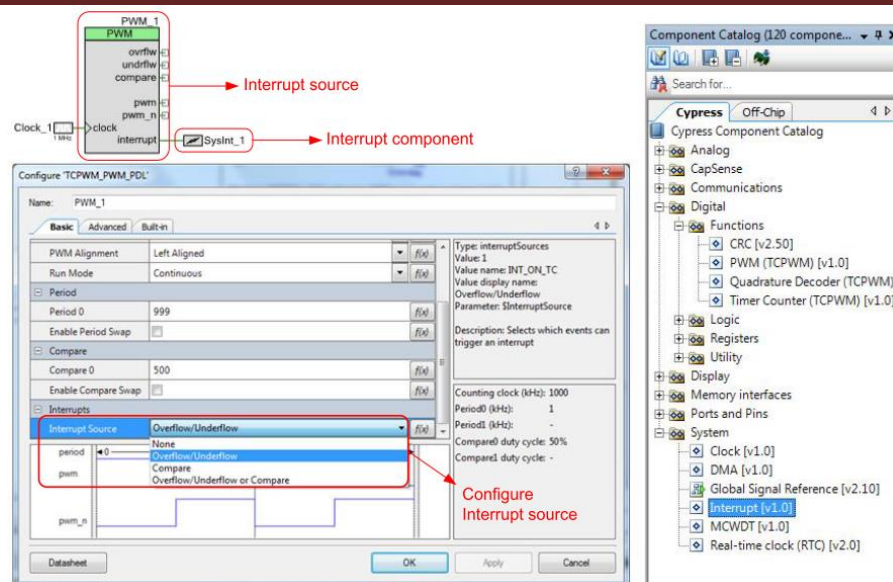


Рис. 4.2. TopDesign з компонентою переривань SysInt_1

Деякі периферійні пристрої не мають зовнішнього виводу переривання (наприклад, SCB має вбудовані переривання) або можуть мати можливість його відкрити (наприклад, UART).

Компонента переривання має дві опції, які налаштовуються (рис. 4.3).

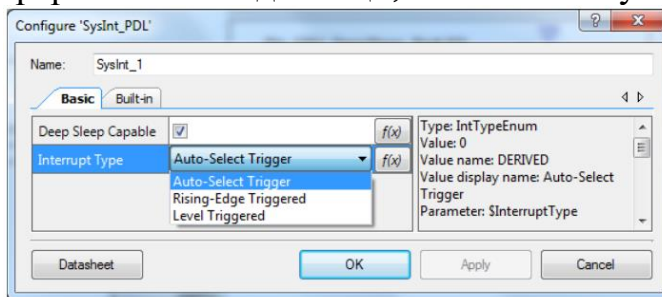


Рис. 4.3. Вигляд вікна конфігурування переривань SysInt

Deep Sleep Capable

Встановіть цей прапорець, якщо хочете, щоб переривання було призначено лінії *IRQn* ЦП, яка підтримує функцію "глибокого сну". Ви повинні переконатися, що джерело переривання також активне і здатне подавати сигнал переривання під час "глибокого сну", у разі відсутності якого середовище PSoC Creator видасть помилку під час збірки проекту. Ця опція є важливою лише у випадку, якщо переривання призначено для ядра CM0+, у якого є 8 (IRQ 0-7) слотів режиму "глибокого сну" для маршрутизації. Цей прапорець призначений тільки для вказівки щодо автоматичного призначення IRQ для переривання і може бути відмінено ручним призначенням у вікні CyDWR. Для ядра CM4, якщо джерело переривання підтримує режим "глибокого сну" (IRQ 0-40), відключення прапорця (заборона) не впливає на функцію "глибокого сну" переривання.

Тип переривання.

У конфігурації компоненти переривання доступні три параметри для типу переривання: переривання з автоматичним вибором, переривання по

передньому (додатному) фронту та переривання по рівню. Вибір конкретного параметра залежить від джерела переривання (фіксована функція або UDB/DSI) та вимог додатку. У більшості випадків залиште опцію автоматичний вибір (Auto-Select), щоб дозволити середовищу PSoC Creator отримувати тип переривання від природи джерела переривання.

Виберіть тільки переривання по рівню для джерел переривань з фіксованою функцією. Виберіть тільки переривання по рівню та переривання по передньому фронті для джерел UDB.

Використання згенерованого коду середовищем PSoC Creator та PDL.

Збірка проекту генерує код для використання в додатку. Папка Pins and Interrupts містить файли з кодом, сформованим за допомогою інформації, що вводиться на вкладці Interrupts в CyDWR.

cyfitter_sysint.h містить макроси з інформацією про номер переривання, призначення його процесору та пріоритет.

cyfitter_sysint_cfg.c/h оголошує та попередньо заповнює екземпляри структури конфігурації SysInt PDL, використовуючи інформацію CyDWR.

Структура конфігурації для кожного переривання визначається умовно на основі призначення ЦП.

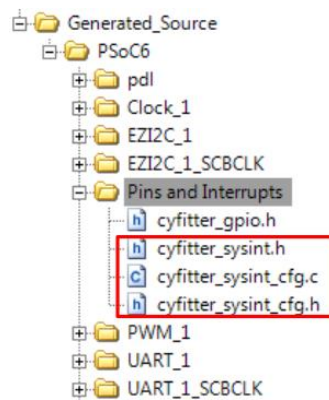


Рис. 4.4. Згенеровані файли

Кроки для дозволу переривань в коді додатку подібні до описаних раніше (PDL), але їх менше.

1. Виклик `__enable_irq()` API для дозволу глобальних переривань.
2. Виклик `Cy_SysInt_Init(&SysInt_1_cfg, ISR_1_handler)`.

де `SysInt_1_cfg` - ім'я автоматично сформованої структури з файлу `cyfitter_sysint_cfg.c`. `ISR_1_handler` - ім'я обробника переривання, який виконується, коли запускається переривання. Функція обробника може знаходитись у відповідній головній програмі `main.c` ЦП, якому призначено переривання. Якщо обробник існує поза `main.c`, цей файл повинен бути скомпільований і зв'язаний з виконуваним файлом для ЦП, який обробляє ISR.

Цей крок налаштовує переривання (маршрутизація, пріоритет та призначення обробника переривання), але не дозволяє його.

3. Виклик `NVIC_ClearPendingIRQ(SysInt_1_cfg.intrSrc)` очищає будь-які очікувані переривання.

4. Виклик `NVIC_EnableIRQ(SysInt_1_cfg.intrSrc)` дозволяє переривання.

Розглянемо приклад проекту, який демонструє використання GPIO, налаштованого як вхідний вивід, для генерування переривань ядром CM4 в мікроконтролері PSoC 6. Сигнал GPIO перериває програму, виконувану ЦП, і виконує визначену користувачем програму обробки переривання (ISR). Переривання GPIO виступає джерелом пробудження, щоб розбудити ЦП з "глибокого сну".

Операції:

1. Підключити стенд до USB-порту комп'ютера.
2. Створити проект і запрограмувати його для мікроконтролера PSoC 6. Для цього вибрати `Debug>Program`.
3. Переконатися, що світлодіод блимає шість разів, а потім вимикається, вказуючи на те, що процесор перейшов у режим глибокого сну.
4. Натиснути кнопку користувача (SW2), підключену до порту P0[4], щоб викликати переривання. Це повинно спричинити пробудження пристрою, що призведе до того, що світлодіод знову почне блимати з новою частотою (більша частота блимання свідчить про те, що ISR виконано). Світлодіод блимає шість разів, і пристрій знову переходить у режим "глибокого сну".
5. Повторне натискання кнопки повторює цикл пробудження, і світлодіод продовжує блимати з початковою частотою 1 Гц. З кожним перериванням і виконанням ISR частота блимання світлодіода чергується між 1 Гц і 2 Гц.

Приклад проекту з використанням переривань МК PSoC 6.

В цьому проекті використовується переривання GPIO (P0.4) для пробудження процесора CM4 з "глибокого сну". Червоний світлодіод RGB LED (GPIO P0.3) використовується для індикації поточного стану процесора. Блімаючий індикатор вказує на те, що центральний процесор активний. Після шести послідовних блимань ЦП переходить в режим "глибокого сну". Світлодіод RGB LED перестає блимати та вимкнеться, щоб індикувати, що процесор знаходиться в режимі "глибокого сну".

Вивід GPIO (P0.4) налаштований як вхідний, під'єднаний ззовні до кнопки SW2, сконфігурований для генерації переривання при натисканні цієї кнопки. Переривання викликає наступні дії:

1. Генерує сигнал, який виводить процесор з режиму "глибокого сну".
2. Виконує програму обробки переривання (ISR).

Коли виконується ISR, оновлюється прапорець, який використовується для зміни швидкості блимання світлодіода. З кожним натисканням кнопки світлодіод по черзі блимає з періодами 500 мсек та 1 сек.

На рис. 4.6 зображено схему цього проекту в PSoC Creator. Проект використовує компоненти GPIO, Global Signal Reference та SysInt.

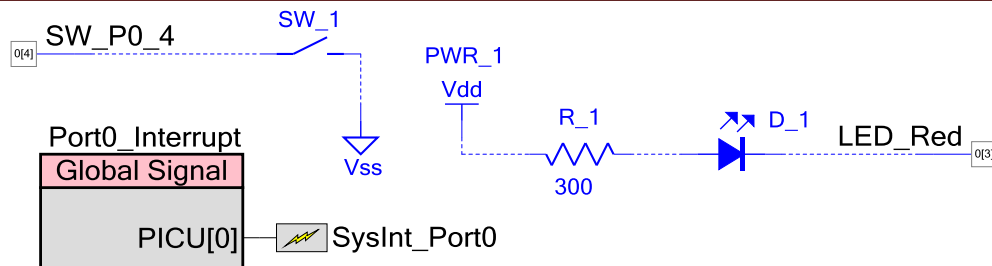


Рис. 4.5. Схема проекту для переривань GPIO в PSoC Creator 4.4

Компонента Global Signal Reference використовується для маршрутизації переривання порту з вхідного виводу. Переривання портів доступні в режимі "живлення глибокого сну" і, таким чином, можуть розбудити процесор з режиму "глибокого сну". Компоненту SysInt також можна підключити безпосередньо до виводу, як показано на рис. 4.6. Це призводить до того, що вивід використовується як джерело переривання UDB. Однак цей сигнал переривання направляєється через цифрове системне з'єднання (DSI) і недоступний під час "глибокого сну" і не може розбудити процесор.



Рис. 4.6. Пряме під'єднання вхідного виводу до компоненти SysInt

На рис. 4.7, ... , рис. 4.9 зображені параметри, які не використовуються за замовчуванням в цьому проекті.

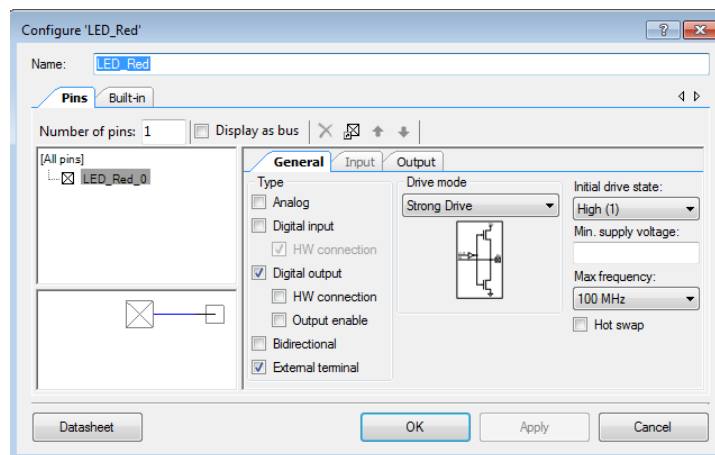


Рис. 4.7. Конфігурація виводу GPIO для LED_Red

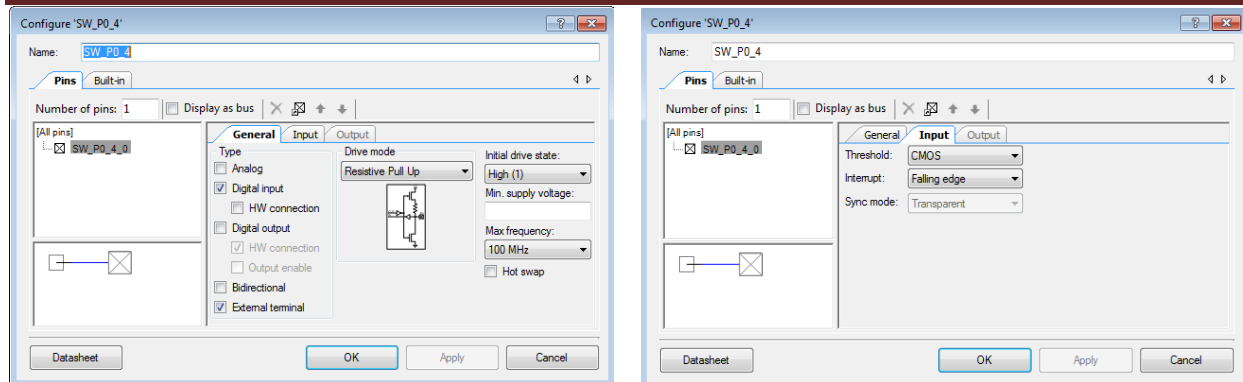


Рис. 4.8. Конфігурація вхідного виводу GPIO для кнопки SW_P0_4

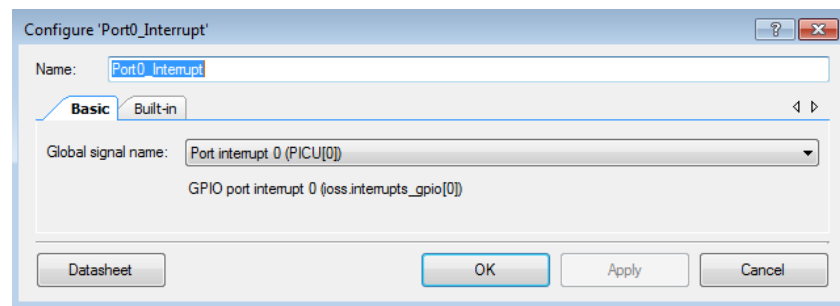


Рис. 4.9. Конфігурація Global Signal Reference

На рис. 4.10 показана конфігурація переривання для цього проекту. Відмітимо, що компонента SysInt призначена ядру CM4 через прапорець. Пріоритет переривань залишається за замовчуванням, оскільки немає інших переривань.

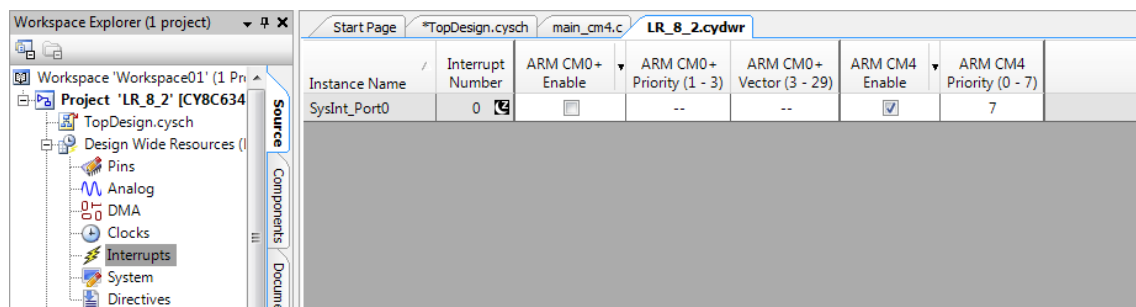


Рис. 4.10. Конфігурація системи переривань

Код програми проекту:

```
#include "project.h"

/* Макроси для налаштування проекту */
#define DELAY_SHORT          (500)      /* msec */
#define DELAY_LONG           (1000)     /* msec */

#define LED_BLINK_COUNT      (6)

#define LED_ON                (0u)
```

```
#define LED_OFF (1u)
```

```
// Глобальні змінні  
uint32_t interrupt_flag = false;
```

Функція: void Isr_switch(void). Ця функція виконується, коли спрацьовує переривання GPIO.

```
void Isr_switch(void)
{
    /* Очищення переривання по виводу SW_P0_4 */
    Cy_GPIO_ClearInterrupt(SW_P0_4_PORT, SW_P0_4_NUM);
    NVIC_ClearPendingIRQ(SysInt_Port0_cfg.intrSrc);
    /* Встановити прапорець переривання */
    interrupt_flag = true;
}

int main(void)
{
    uint32_t count = 0;
    uint32_t delayMs = DELAY_LONG;

    /* Дозвіл глобальних переривань */
    __enable_irq();

    /* Ініціалізація та дозвіл переривань GPIO */
    Cy_SysInt_Init(&SysInt_Port0_cfg, Isr_switch);
    NVIC_ClearPendingIRQ(SysInt_Port0_cfg.intrSrc);
    NVIC_EnableIRQ(SysInt_Port0_cfg.intrSrc);

    for(;;)
    {
        /* Зміна статусу переривання */
        if(interrupt_flag == true)
        {
            interrupt_flag = false;
            /* Зміна частоти блимання LED */
            if(DELAY_LONG == delayMs)
            {
                delayMs = DELAY_SHORT;
            }
            else
            {
                delayMs = DELAY_LONG;
            }
        }

        /* Цикл блимання світлодіодом LED LED_BLINK_COUNT разів */
        for (count = 0; count < LED_BLINK_COUNT; count++)
        {
            Cy_GPIO_Write(LED_Red_PORT, LED_Red_NUM, LED_ON);
        }
    }
}
```

```
Cy_SysLib_Delay(delayMs);  
Cy_GPIO_Write(LED_Red_PORT, LED_Red_NUM, LED_OFF);  
Cy_SysLib_Delay(delayMs);  
}  
/* Перехід в режим "глибокого сну" */  
Cy_SysPm_DeepSleep(CY_SYSPM_WAIT_FOR_INTERRUPT);  
}  
}
```

Приклад 2. Розглянемо інший проект, в якому виконується генерація періодичних переривань за допомогою екземпляра компоненти TCPWM в режимі таймера/лічильника мікроконтролера PSoC 6.

У цьому проекті екземпляр компоненти TCPWM налаштовується в режимі неперервного сумуючого лічильника для створення періодичного переривання. Процесор переходить у сплячий режим. Він прокидається кожного разу, коли відбувається переривання, і повертається в режим сну після обслуговування переривання. Обробник переривання просто перемикає світлодіод. Можна змінити період таймера, змінивши макрос `TIMER_PERIOD_MSEC` у файлі `main_cm4.c`. На рис. 4.11 зображено схему, отриману в середовищі PSoC Creator для цього проекту.

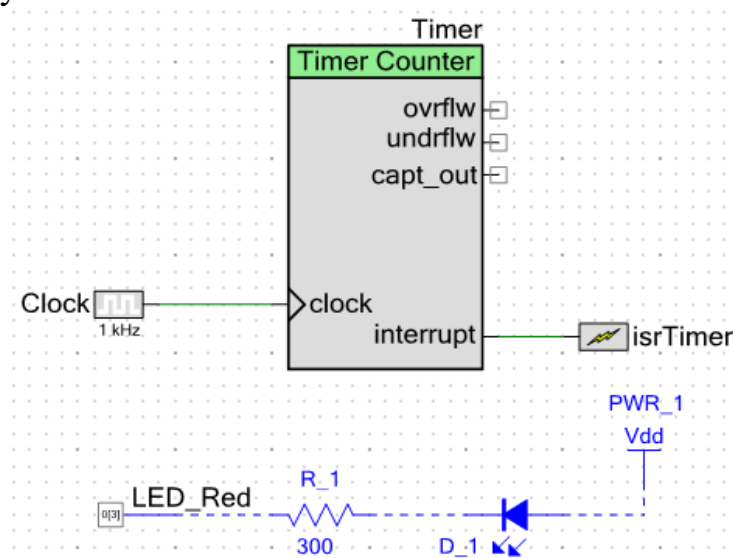


Рис. 4.11. Схема апаратної частини проекту з використанням компоненти таймер/лічильник

Програмна частина до проекту, апаратна частина якого зображена на рис. 4.11.

```
// Code Project LR_4_2  
  
#include "project.h"  
#define TIMER_PERIOD_MSEC 1500u /* Період таймера в ms */  
  
/* Функція обробника переривання по таймеру, яка просто перемикає світлодіод.*/
```

Методичні вказівки з курсу “Вбудовані системи опрацювання даних та управління на основі нейронних мереж”, 2025 р. Рабик В.

```
void TimerInterruptHandler(void)
{
    /* Очистка прерывания при переполнении счетчика таймера */
    Cy_TCPWM_ClearInterrupt(Timer_HW, Timer_CNT_NUM, CY_TCPWM_INT_ON_TC);

    /* Переключение светодиода LED */
    Cy_GPIO_Inv(LED_Red_PORT, LED_Red_NUM);
}

int main(void)
{
    Cy_SysInt_Init(&isrTimer_cfg, TimerInterruptHandler);
    NVIC_EnableIRQ(isrTimer_cfg.intrSrc);
    /* Дозвіл переривань ядра */
    _enable_irq();
    /* Дозвіл глобальних переривань */

    /* Запуск компонента TCPWM в режиме таймера/счетчика. */
    (void)Cy_TCPWM_Counter_Init(Timer_HW, Timer_CNT_NUM, &Timer_config);
    Cy_TCPWM_Enable_Multiple(Timer_HW, Timer_CNT_MASK);

    /* Дозвіл экземпляра таймера/счетчика */
    /* Встановлення періоду таймера в мілісекундах. */
    Cy_TCPWM_Counter_SetPeriod(Timer_HW, Timer_CNT_NUM, TIMER_PERIOD_MSEC-1);

    /* Програмний запуск экземпляра счетчика.
     * Це необхідно виконати, якщо жодний інший апаратний вхідний сигнал не
     * підключений до компонента, щоб запустити її. */
    Cy_TCPWM_TriggerStart(Timer_HW, Timer_CNT_MASK);

    for(;;)
    {
        /* Переведення процесора в сплячий режим для економії енергії.
         * Процесор прокидається кожного разу, коли відбувається переривання по
         * таймеру
         * і знову переходить в режим сну після обслуговування переривання ISR.
         */
        Cy_SysPm_Sleep(CY_SYSPM_WAIT_FOR_INTERRUPT);
    }
}
```

Налаштування параметрів компонента проекту (рис. 4.12).

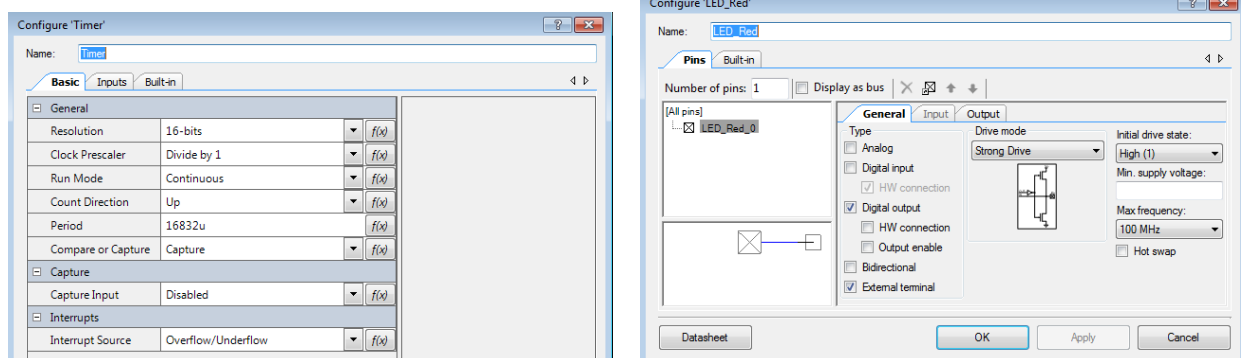


Рис. 4.12. Налаштування параметрів компонент проекту

Функції, що використовуються в цих проектах.

`Void NVIC_ClearPendingIRQ (IRQn_Type IRQn);`

Ця функція видаляє стан очікування заданого переривання IRQn. IRQn не може бути від’ємним числом.

Параметри:

[in] IRQn Interrupt number

Зауваження:

- реєстри, які керують станом переривань: SETPEND, CLRPEND.
- переривання може мати статус очікування, хоча воно є неактивним.

`Void NVIC_EnableIRQ (IRQn_Type IRQn);`

Ця функція дозволяє вказане переривання IRQn для конкретного пристрою. IRQn не може бути від’ємним числом.

Завдання.

1. Реалізувати проект засвічування світлодіодів LED8 та LED9 стенду PSoC 6 BLE PIONER KIT з використанням переривань виводу GPIO, до якого під’єднано кнопку SW_2. Алгоритм засвічування світлодіодів: при включенні живлення стенду світлодіод LED8 блимає з частотою 1.25 Гц 6 разів. Після цього процесор переходить в режим "глибокого сну". З цього режиму його виводить натискання на кнопку SW_2. При настанні переривання починає блимати 4 рази світлодіод LED9 з частотою 2 Гц. Процесор знову переходить в режим "глибокого сну", з якого його можна вивести натисканням на кнопку SW_2. Але при цьому весь час змінюється частота і кількість блимань світлодіода LED9 (1 Гц 4 рази та 2 Гц 6 разів).

2. Реалізувати проект, в якому засвічується синій світлодіод RGB LED стенду PSoC 6 BLE PIONEER KIT з періодом 1.6 сек (0.8 сек – ON, 0.8 сек – OFF), якщо кнопка SW2 не натиснута. При кожному натисканні на кнопку SW2 засвічуються послідовно зелений світлодіод RGB LED з періодом 1.8 сек (0.9 сек – ON, 0.9 сек – OFF) або червоний світлодіод з періодом 1.0 сек (0.5 сек – ON, 0.5 сек – OFF). Використати переривання виводу GPIO, до якого під’єднано кнопку SW_2. Підпрограма обробки переривання реалізує перемикання зеленого та червоного світлодіодів та вибір періоду їх засвічування.